

mvp_report

December 3, 2025

1 LLM Cost Decomposition Analysis

Comprehensive analysis of multi-stage pipelines and agentic workflows.

1.1 Analyses:

1. Pipeline-level cost comparison
2. Stage-level cost attribution
3. Multi-model hybrid analysis
4. Agentic patterns (ReAct iterations, multi-turn context growth)
5. Streaming metrics (TTFT, throughput)
6. Cost-quality tradeoffs

```
[18]: import pandas as pd
import numpy as np
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
from scipy import stats
import json

import sys
sys.path.insert(0, '..')

from src.db import (
    get_runs,
    get_stages,
    get_quality_scores,
    get_runs_with_quality,
    get_pipeline_summary,
    get_stage_summary,
    get_cost_by_stage_type,
    get_cost_by_model,
    get_iteration_analysis,
    get_context_growth_analysis,
    get_streaming_analysis,
)
from src.cost_calculator import format_cost
```

1.2 1. Load Data

```
[19]: runs_df = get_runs(success_only=True)
      stages_df = get_stages()
      quality_df = get_quality_scores()

      print(f"Total pipeline runs: {len(runs_df)}")
      print(f"Total stage executions: {len(stages_df)}")
      print(f"Quality evaluations: {len(quality_df)}")

      if len(runs_df) > 0:
          print(f"\nPipeline types: {runs_df['pipeline_type'].unique().tolist()}")
          print(f"Workflows: {runs_df['workflow'].unique().tolist()}")
```

Total pipeline runs: 591

Total stage executions: 1602

Quality evaluations: 590

Pipeline types: ['linear', 'self_correcting', 'multiturn', 'react']

Workflows: ['document', 'self_correcting', 'multiturn', 'react', 'context', 'verbosity']

```
[20]: # Pipeline summary
      pipeline_summary = get_pipeline_summary()
      if not pipeline_summary.empty:
          display(pipeline_summary)
```

	workflow	pipeline	pipeline_type \
0	context	context_long	linear
1	context	context_long	linear
2	context	context_short	linear
3	context	context_short	linear
4	document	doc_analysis_hybrid	linear
5	document	doc_analysis_hybrid	linear
6	document	doc_analysis_iterative	linear
7	document	doc_analysis_iterative	linear
8	document	doc_analysis_simple	linear
9	document	doc_analysis_simple	linear
10	document	doc_analysis_thorough	linear
11	document	doc_analysis_thorough	linear
12	multiturn	multiturn_3	multiturn
13	multiturn	multiturn_3	multiturn
14	multiturn	multiturn_5	multiturn
15	multiturn	multiturn_5	multiturn
16	react	react_hybrid	react
17	react	react_hybrid	react
18	react	react_research	react
19	react	react_research	react
20	self_correcting	self_correcting	self_correcting

21	self_correcting	self_correcting	self_correcting
22	self_correcting	self_correcting_hybrid	self_correcting
23	self_correcting	self_correcting_hybrid	self_correcting
24	verbosity	hybrid_cot	linear
25	verbosity	hybrid_cot	linear
26	verbosity	verbosity_concise	linear
27	verbosity	verbosity_concise	linear
28	verbosity	verbosity_cot	linear
29	verbosity	verbosity_cot	linear

	model	run_count	mean_cost	mean_input_tokens \
0	gemini-2.5-flash	20	0.001119	1356.000000
1	gemini-2.5-pro	20	0.009191	1351.550000
2	gemini-2.5-flash	20	0.000103	147.800000
3	gemini-2.5-pro	20	0.000923	158.850000
4	gemini-2.5-flash	20	0.017509	3769.400000
5	gemini-2.5-pro	20	0.016445	3662.250000
6	gemini-2.5-flash	20	0.014329	4278.750000
7	gemini-2.5-pro	20	0.029989	3451.550000
8	gemini-2.5-flash	20	0.002155	2018.550000
9	gemini-2.5-pro	20	0.014357	1760.350000
10	gemini-2.5-flash	20	0.015684	5859.650000
11	gemini-2.5-pro	20	0.046544	5350.550000
12	gemini-2.5-flash	20	0.003987	5262.700000
13	gemini-2.5-pro	20	0.026625	4095.800000
14	gemini-2.5-flash	20	0.005521	13411.350000
15	gemini-2.5-pro	20	0.045216	12779.150000
16	gemini-2.5-flash	20	0.010125	1951.600000
17	gemini-2.5-pro	20	0.010852	2424.450000
18	gemini-2.5-flash	20	0.000481	74.000000
19	gemini-2.5-pro	20	0.000567	74.000000
20	gemini-2.5-flash	20	0.001656	2117.950000
21	gemini-2.5-pro	20	0.001850	2391.600000
22	gemini-2.5-flash	19	0.004840	1800.315789
23	gemini-2.5-pro	20	0.004697	1715.100000
24	gemini-2.5-flash	20	0.009479	2680.400000
25	gemini-2.5-pro	20	0.009586	2672.200000
26	gemini-2.5-flash	20	0.000030	17.400000
27	gemini-2.5-pro	20	0.000250	17.400000
28	gemini-2.5-flash	20	0.002601	2350.750000
29	gemini-2.5-pro	12	0.022204	2555.333333

	mean_output_tokens	mean_latency_ms	avg_stages	avg_iterations \
0	1526.500000	41219.350000	3.000000	1.000000
1	1500.350000	65015.650000	3.000000	1.000000
2	134.000000	9689.550000	2.000000	1.000000
3	144.850000	20541.000000	2.000000	1.000000
4	8416.400000	101544.000000	3.000000	1.000000

5	7681.650000	95295.400000	3.000000	1.000000
6	6834.850000	98324.400000	3.000000	1.000000
7	5134.950000	118429.150000	3.000000	1.000000
8	3086.950000	37496.300000	2.000000	1.000000
9	2431.350000	61723.250000	2.000000	1.000000
10	10485.350000	153281.800000	4.000000	1.000000
11	7971.100000	175449.650000	4.000000	1.000000
12	5329.300000	65315.950000	3.000000	1.000000
13	4301.100000	95458.900000	3.000000	1.000000
14	5849.100000	80855.800000	5.000000	1.000000
15	5848.350000	135069.800000	5.000000	1.000000
16	1985.400000	45660.950000	2.750000	1.850000
17	2172.250000	49287.050000	2.950000	1.950000
18	783.250000	8725.350000	1.000000	1.000000
19	926.450000	9957.050000	1.000000	1.000000
20	2230.500000	32877.300000	2.300000	1.150000
21	2485.400000	39227.550000	2.400000	1.200000
22	1899.789474	41377.578947	2.210526	1.105263
23	1887.100000	38460.100000	2.100000	1.050000
24	4203.900000	72160.500000	3.000000	1.000000
25	4085.900000	72033.250000	3.000000	1.000000
26	45.800000	3827.800000	1.000000	1.000000
27	45.600000	10531.800000	1.000000	1.000000
28	3747.650000	54087.750000	3.000000	1.000000
29	3802.083333	97286.583333	3.000000	1.000000

	avg_turns	mean_ttft_ms	mean_quality_score
0	1.0	None	72.314500
1	1.0	None	72.935500
2	1.0	None	71.299000
3	1.0	None	74.192000
4	1.0	None	80.593500
5	1.0	None	82.265000
6	1.0	None	85.283000
7	1.0	None	87.066500
8	1.0	None	86.611500
9	1.0	None	88.690500
10	1.0	None	83.066316
11	1.0	None	84.900500
12	3.0	None	56.636000
13	3.0	None	57.679000
14	5.0	None	64.897000
15	5.0	None	85.606000
16	1.0	None	89.063500
17	1.0	None	90.083000
18	1.0	None	91.980000
19	1.0	None	89.977000
20	1.0	None	85.835000

21	1.0	None	86.065500
22	1.0	None	86.293684
23	1.0	None	85.027000
24	1.0	None	87.139500
25	1.0	None	87.637000
26	1.0	None	87.608000
27	1.0	None	87.111000
28	1.0	None	87.936000
29	1.0	None	87.767500

1.3 2. Pipeline Cost Comparison

```
[21]: if len(runs_df) > 0:
    fig = px.box(
        runs_df,
        x='pipeline',
        y='total_cost',
        color='model',
        facet_row='pipeline_type',
        title='Cost Distribution by Pipeline Type',
        labels={'total_cost': 'Total Cost ($)', 'pipeline': 'Pipeline'}
    )
    fig.update_layout(height=800)
    fig.show()
```

1.4 3. Multi-Model Hybrid Analysis

Compare hybrid pipelines (using different models per stage) vs single-model pipelines.

```
[22]: # Cost breakdown by model used
cost_by_model = get_cost_by_model()
if not cost_by_model.empty:
    print("Cost by Model:")
    display(cost_by_model)

    fig = px.pie(
        cost_by_model,
        values='total_cost',
        names='model',
        title='Total Cost Distribution by Model'
    )
    fig.show()
```

Cost by Model:

	model	stage_count	total_cost	avg_cost	avg_latency_ms	\
0	gemini-2.5-pro	722	5.522902	0.007649	30879.541551	
1	gemini-2.5-flash	854	0.872919	0.001022	18137.601874	

```

    avg_ttft_ms
0          None
1          None

```

```

[23]: # Compare hybrid vs non-hybrid CoT
if len(runs_df) > 0:
    cot_pipelines = runs_df[runs_df['pipeline'].str.contains('cot', case=False)]

    if len(cot_pipelines) > 0:
        cot_comparison = cot_pipelines.groupby('pipeline').agg({
            'total_cost': ['mean', 'std', 'count'],
            'total_latency_ms': 'mean',
        }).round(6)

        print("\nCoT Pipeline Comparison (including hybrid):")
        display(cot_comparison)

```

CoT Pipeline Comparison (including hybrid):

	total_cost		total_latency_ms	
	mean	std count	mean	
pipeline				
hybrid_cot	0.009532	0.001693 40	72096.8750	
verbosity_cot	0.009952	0.009800 32	70287.3125	

1.5 4. Stage-Level Cost Attribution

```

[24]: stage_summary = get_stage_summary()
if not stage_summary.empty:
    print("Stage Type Summary:")
    display(stage_summary)

```

Stage Type Summary:

	stage_type	model	count	avg_cost	total_cost \
0	action	gemini-2.5-flash	38	0.000379	0.014400
1	conversation	gemini-2.5-flash	160	0.001188	0.190157
2	conversation	gemini-2.5-pro	149	0.009643	1.436816
3	critique	gemini-2.5-flash	20	0.000721	0.014425
4	critique	gemini-2.5-pro	92	0.008636	0.794535
5	evaluation	gemini-2.5-flash	20	0.000456	0.009118
6	evaluation	gemini-2.5-pro	60	0.007380	0.442804
7	extraction	gemini-2.5-flash	120	0.000598	0.071707
8	extraction	gemini-2.5-pro	80	0.004652	0.372151
9	generation	gemini-2.5-flash	219	0.000924	0.202288
10	generation	gemini-2.5-pro	132	0.008628	1.138840
11	refinement	gemini-2.5-flash	150	0.002115	0.317319
12	refinement	gemini-2.5-pro	52	0.013929	0.724320

13	summarization	gemini-2.5-flash	40	0.000175	0.007016
14	summarization	gemini-2.5-pro	40	0.001477	0.059065
15	thinking	gemini-2.5-flash	40	0.000524	0.020961
16	thinking	gemini-2.5-pro	75	0.005402	0.405145
17	validation	gemini-2.5-flash	47	0.000543	0.025528
18	validation	gemini-2.5-pro	42	0.003553	0.149226

	avg_input_tokens	avg_output_tokens	avg_latency_ms	avg_ttft_ms	\
0	548.052632	494.684211	7198.157895	None	
1	2334.256250	1397.300000	18271.468750	None	
2	2265.093960	1362.342282	30943.449664	None	
3	1468.000000	834.950000	17224.700000	None	
4	1603.086957	1326.478261	37719.380435	None	
5	401.550000	659.500000	20714.400000	None	
6	1591.683333	1078.100000	37017.433333	None	
7	329.133333	913.616667	13424.466667	None	
8	314.100000	851.837500	23487.600000	None	
9	351.908676	1451.470320	18906.438356	None	
10	800.590909	1525.378788	35890.409091	None	
11	1641.273333	3115.506667	30494.493333	None	
12	1409.519231	2433.480769	45271.403846	None	
13	297.625000	217.875000	6822.475000	None	
14	297.825000	220.875000	15339.325000	None	
15	74.000000	854.850000	9341.200000	None	
16	889.266667	858.066667	21672.400000	None	
17	1566.191489	513.617021	11948.893617	None	
18	1494.523810	336.976190	18656.380952	None	

	avg_throughput
0	None
1	None
2	None
3	None
4	None
5	None
6	None
7	None
8	None
9	None
10	None
11	None
12	None
13	None
14	None
15	None
16	None
17	None
18	None

```
[25]: # Cost by stage type visualization
cost_by_stage = get_cost_by_stage_type()

if not cost_by_stage.empty:
    fig = px.bar(
        cost_by_stage,
        x='stage_type',
        y='total_cost',
        color='model',
        barmode='group',
        title='Total Cost by Stage Type and Model',
        labels={'total_cost': 'Total Cost ($)', 'stage_type': 'Stage Type'}
    )
    fig.show()
```

```
[26]: # Sunburst for hierarchical cost view
if not cost_by_stage.empty:
    fig = px.sunburst(
        cost_by_stage,
        path=['pipeline_type', 'pipeline', 'stage_type'],
        values='total_cost',
        title='Cost Decomposition Hierarchy'
    )
    fig.show()
```

1.6 5. Agentic Pattern Analysis

1.6.1 5.1 ReAct Iteration Analysis

```
[27]: iteration_analysis = get_iteration_analysis()

if not iteration_analysis.empty:
    print("Iteration Analysis (ReAct & Self-Correcting):")
    display(iteration_analysis)

    # Cost vs iterations
    fig = px.bar(
        iteration_analysis,
        x='iterations',
        y='avg_cost',
        color='pipeline',
        barmode='group',
        title='Average Cost by Number of Iterations',
        labels={'avg_cost': 'Average Cost ($)', 'iterations': 'Iterations'}
    )
    fig.show()
else:
    print("No agentic pipeline data available.")
```

Iteration Analysis (ReAct & Self-Correcting):

	pipeline	pipeline_type	iterations	termination_reason \
0	multiturn_3	multiturn	1	None
1	multiturn_5	multiturn	1	None
2	react_hybrid	react	1	confidence_reached
3	react_hybrid	react	2	confidence_reached
4	react_hybrid	react	3	confidence_reached
5	react_hybrid	react	4	confidence_reached
6	react_hybrid	react	5	confidence_reached
7	react_hybrid	react	5	max_iterations
8	react_research	react	1	confidence_reached
9	self_correcting	self_correcting	1	validation_passed
10	self_correcting	self_correcting	2	validation_passed
11	self_correcting	self_correcting	3	validation_passed
12	self_correcting_hybrid	self_correcting	1	validation_passed
13	self_correcting_hybrid	self_correcting	2	validation_passed

	run_count	avg_cost	avg_stages
0	40	0.015306	3.0
1	40	0.025368	5.0
2	20	0.006527	1.0
3	11	0.009339	3.0
4	5	0.013865	5.0
5	1	0.026562	7.0
6	1	0.041298	9.0
7	2	0.024552	10.0
8	40	0.000524	1.0

9	34	0.001382	2.0
10	5	0.003496	4.0
11	1	0.005660	6.0
12	36	0.004414	2.0
13	3	0.008995	4.0

```
[28]: # Termination reason analysis
if not iteration_analysis.empty and 'termination_reason' in iteration_analysis.
    columns:
        term_counts = iteration_analysis.groupby(['pipeline', '
        'termination_reason'])['run_count'].sum().reset_index()

    fig = px.bar(
        term_counts,
        x='pipeline',
        y='run_count',
        color='termination_reason',
        title='Termination Reasons by Pipeline',
        labels={'run_count': 'Run Count'}
    )
    fig.show()
```

1.6.2 5.2 Multi-Turn Context Growth Analysis

```
[29]: context_growth = get_context_growth_analysis()

if not context_growth.empty:
    print("Context Token Growth by Turn:")
    display(context_growth)

    fig = px.line(
        context_growth,
        x='turn',
        y='avg_context_tokens',
        color='pipeline',
        markers=True,
        title='Context Token Growth Across Conversation Turns',
        labels={'avg_context_tokens': 'Average Context Tokens', 'turn': 'Turn_
        Number'})
    )
    fig.show()

    # Cost per turn
    fig2 = px.line(
        context_growth,
        x='turn',
        y='avg_cost_per_turn',
```

```

        color='pipeline',
        markers=True,
        title='Cost Growth Per Turn',
        labels={'avg_cost_per_turn': 'Average Cost ($)', 'turn': 'Turn Number'}
    )
    fig2.show()
else:
    print("No multi-turn conversation data available.")

```

Context Token Growth by Turn:

	pipeline	turns	turn	avg_context_tokens	avg_cost_per_turn
0	multiturn_3	3	1	5.800	0.003468
1	multiturn_3	3	2	1230.175	0.005801
2	multiturn_3	3	3	3443.275	0.006037
3	multiturn_5	5	1	5.950	0.003546
4	multiturn_5	5	2	1277.550	0.004055
5	multiturn_5	5	3	2449.050	0.005849
6	multiturn_5	5	4	4002.775	0.006628
7	multiturn_5	5	5	5359.925	0.005290

1.7 6. Streaming Metrics Analysis (TTFT & Throughput)

```

[30]: streaming_analysis = get_streaming_analysis()

if not streaming_analysis.empty:
    print("Streaming Metrics:")
    display(streaming_analysis)

    # TTFT comparison
    fig = px.bar(
        streaming_analysis,
        x='stage_type',
        y='avg_ttft_ms',
        color='model',
        barmode='group',
        error_y=streaming_analysis['max_ttft_ms'] -
streaming_analysis['avg_ttft_ms'],
        title='Time to First Token by Stage Type',
        labels={'avg_ttft_ms': 'Average TTFT (ms)', 'stage_type': 'Stage Type'}
    )
    fig.show()

    # Throughput comparison
    fig2 = px.bar(
        streaming_analysis,
        x='stage_type',
        y='avg_throughput',

```

```

        color='model',
        barmode='group',
        title='Token Throughput by Stage Type',
        labels={'avg_throughput': 'Tokens/Second', 'stage_type': 'Stage Type'}
    )
    fig2.show()
else:
    print("No streaming data available. Run experiments with --streaming flag.")

```

No streaming data available. Run experiments with --streaming flag.

```

[31]: # TTFT vs Total Latency scatter
if not streaming_analysis.empty:
    fig = px.scatter(
        streaming_analysis,
        x='avg_ttft_ms',
        y='avg_total_latency_ms',
        color='model',
        symbol='stage_type',
        size='count',
        title='TTFT vs Total Latency',
        labels={
            'avg_ttft_ms': 'Time to First Token (ms)',
            'avg_total_latency_ms': 'Total Latency (ms)'
        }
    )
    fig.show()

```

1.8 7. Cost-Quality Tradeoff Analysis

```

[32]: runs_with_quality = get_runs_with_quality()

if len(runs_with_quality) > 0 and 'combined_score' in runs_with_quality.columns:
    quality_data = runs_with_quality.dropna(subset=['combined_score'])

    if len(quality_data) > 0:
        fig = px.scatter(
            quality_data,
            x='total_cost',
            y='combined_score',
            color='pipeline',
            symbol='model',
            size='num_stages',
            hover_data=['iterations', 'turns', 'total_latency_ms'],
            title='Cost vs Quality Tradeoff',
            labels={'total_cost': 'Total Cost ($)', 'combined_score': 'Quality_
↳Score'}

```

```

    )
    fig.show()

    # Quality per dollar
    quality_data = quality_data.copy()
    quality_data['quality_per_millicent'] = quality_data['combined_score'] /
    ↪ (quality_data['total_cost'] * 1000)

    efficiency = quality_data.groupby(['pipeline', 'model']).agg({
        'combined_score': 'mean',
        'total_cost': 'mean',
        'quality_per_millicent': 'mean'
    }).round(4).reset_index().sort_values('quality_per_millicent',
    ↪ ascending=False)

    print("\nCost-Quality Efficiency (higher is better):")
    display(efficiency)
else:
    print("No quality data available. Run with --llm-eval for detailed quality_
    ↪ analysis.")

```

Cost-Quality Efficiency (higher is better):

	pipeline	model	combined_score	total_cost \
26	verbosity_concise	gemini-2.5-flash	87.6080	0.0000
2	context_short	gemini-2.5-flash	71.2990	0.0001
27	verbosity_concise	gemini-2.5-pro	87.1110	0.0002
20	react_research	gemini-2.5-flash	91.9800	0.0005
21	react_research	gemini-2.5-pro	89.9770	0.0006
3	context_short	gemini-2.5-pro	74.1920	0.0009
22	self_correcting	gemini-2.5-flash	85.8350	0.0017
0	context_long	gemini-2.5-flash	72.3145	0.0011
23	self_correcting	gemini-2.5-pro	86.0655	0.0019
8	doc_analysis_simple	gemini-2.5-flash	86.6115	0.0022
28	verbosity_cot	gemini-2.5-flash	87.9360	0.0026
24	self_correcting_hybrid	gemini-2.5-flash	86.2937	0.0048
25	self_correcting_hybrid	gemini-2.5-pro	85.0270	0.0047
14	multiturn_3	gemini-2.5-flash	56.6360	0.0040
16	multiturn_5	gemini-2.5-flash	64.8970	0.0055
19	react_hybrid	gemini-2.5-pro	90.0830	0.0109
18	react_hybrid	gemini-2.5-flash	89.0635	0.0101
12	hybrid_cot	gemini-2.5-flash	87.1395	0.0095
13	hybrid_cot	gemini-2.5-pro	87.6370	0.0096
1	context_long	gemini-2.5-pro	72.9355	0.0092
9	doc_analysis_simple	gemini-2.5-pro	88.6905	0.0144
6	doc_analysis_iterative	gemini-2.5-flash	85.2830	0.0143
10	doc_analysis_thorough	gemini-2.5-flash	83.0663	0.0158

5	doc_analysis_hybrid	gemini-2.5-pro	82.2650	0.0164
4	doc_analysis_hybrid	gemini-2.5-flash	80.5935	0.0175
29	verbosity_cot	gemini-2.5-pro	87.7675	0.0222
7	doc_analysis_iterative	gemini-2.5-pro	87.0665	0.0300
15	multiturn_3	gemini-2.5-pro	57.6790	0.0266
17	multiturn_5	gemini-2.5-pro	85.6060	0.0452
11	doc_analysis_thorough	gemini-2.5-pro	84.9005	0.0465

	quality_per_millicent
26	2927.3649
2	702.0266
27	354.4535
20	206.9060
21	180.3748
3	80.4852
22	66.1949
0	65.3917
23	64.5858
8	41.9418
28	35.3283
24	21.7534
25	20.3615
14	14.7923
16	12.1105
19	12.0331
18	11.2400
12	9.4779
13	9.4643
1	8.0059
9	6.3637
6	6.0526
10	5.4921
5	5.0730
4	4.7009
29	4.0123
7	2.9260
15	2.5837
17	1.9870
11	1.8478

1.9 8. Key Findings Summary

```
[33]: if len(runs_df) > 0:
      print("="*60)
      print("KEY FINDINGS")
      print("="*60)
```

```

print(f"\nTotal pipeline executions: {len(runs_df)}")
print(f"Total stage executions: {len(stages_df)}")
print(f"Total cost: {format_cost(runs_df['total_cost'].sum())}")

# By pipeline type
print("\n--- By Pipeline Type ---")
for ptype in runs_df['pipeline_type'].unique():
    ptype_df = runs_df[runs_df['pipeline_type'] == ptype]
    print(f"{ptype}: {len(ptype_df)} runs, avg cost:␣
↪{format_cost(ptype_df['total_cost'].mean())}")

# Most expensive stage type
if len(stages_df) > 0:
    stage_costs = stages_df.groupby('stage_type')['cost'].sum().
↪sort_values(ascending=False)
    print(f"\nMost expensive stage type: {stage_costs.index[0]}")
    print(f" Total spend: {format_cost(stage_costs.iloc[0])}")

# Model comparison
cost_by_model = get_cost_by_model()
if not cost_by_model.empty:
    print("\n--- Model Comparison ---")
    for _, row in cost_by_model.iterrows():
        print(f"{row['model']}: {format_cost(row['total_cost'])} total,␣
↪{format_cost(row['avg_cost'])} avg/stage")

# Agentic insights
react_runs = runs_df[runs_df['pipeline_type'] == 'react']
if len(react_runs) > 0:
    print(f"\n--- ReAct Insights ---")
    print(f"Average iterations: {react_runs['iterations'].mean():.1f}")
    print(f"Average cost: {format_cost(react_runs['total_cost'].mean())}")

multiturn_runs = runs_df[runs_df['pipeline_type'] == 'multiturn']
if len(multiturn_runs) > 0:
    print(f"\n--- Multi-Turn Insights ---")
    print(f"Average turns: {multiturn_runs['turns'].mean():.1f}")
    print(f"Average cost: {format_cost(multiturn_runs['total_cost'].
↪mean())}")
else:
    print("No data available. Run experiments first using:")
    print(" python -m src.experiment --full-suite --iterations 10")

```

=====

KEY FINDINGS

=====

Total pipeline executions: 591
Total stage executions: 1602
Total cost: \$6.40

--- By Pipeline Type ---

linear: 352 runs, avg cost: \$0.0116
self_correcting: 79 runs, avg cost: \$0.003241
multiturn: 80 runs, avg cost: \$0.0203
react: 80 runs, avg cost: \$0.005506

Most expensive stage type: conversation
Total spend: \$1.63

--- Model Comparison ---

gemini-2.5-pro: \$5.52 total, \$0.007649 avg/stage
gemini-2.5-flash: \$0.8729 total, \$0.001022 avg/stage

--- ReAct Insights ---

Average iterations: 1.4
Average cost: \$0.005506

--- Multi-Turn Insights ---

Average turns: 4.0
Average cost: \$0.0203

1.10 9. Export Results

```
[34]: if len(runs_df) > 0:
      runs_df.to_csv('../data/pipeline_runs.csv', index=False)
      stages_df.to_csv('../data/stage_results.csv', index=False)

      if len(quality_df) > 0:
          quality_df.to_csv('../data/quality_scores.csv', index=False)

      print("Results exported to data/ directory")
```

Results exported to data/ directory